

Parliament Insights API

Análise e Projeto de Sistemas de uma Camada de IA Generativa e Backend
sobre Dados de Pesquisa Validados

Documento de Engenharia de Software (SAD – Software Architecture Document) descrevendo requisitos, arquitetura, modelo de dados e design de API de um sistema que consome, sem alterá-los, os artefatos científicos produzidos pelo Trabalho de Conclusão de Curso *Parliament Graph Architecture*.

Felipe Echeverria Vilhalva

Bacharelado em Ciência da Computação — UEMS

25 de setembro de 2026

Sumário

1	Introdução	2
1.1	Contexto	2
1.2	Objetivos	2
1.3	Escopo	2
1.4	Público-Alvo	2
2	Requisitos	3
2.1	Requisitos Funcionais	3
2.2	Requisitos Não-Funcionais	3
3	Análise de Casos de Uso	5
3.1	Diagrama de Casos de Uso	5
3.2	Especificação dos Casos de Uso Principais	5
4	Arquitetura de Software	7
4.1	Visão Geral em Camadas	7
4.2	Descrição das Camadas	7
4.3	Padrões e Decisões Arquiteturais	8
5	Modelo de Dados	9
5.1	Diagrama Entidade-Relacionamento	10
5.2	Dicionário de Dados (resumido)	11
6	Fluxos Principais (Diagramas de Sequência)	12
6.1	Fluxo 1 – Consulta Estruturada (UC01)	12
6.2	Fluxo 2 – Busca RAG (UC02)	12
6.3	Fluxo 3 – Sumarização em Lote (UC03)	13
7	Design de API	14
7.1	Endpoints REST	14
7.2	Exemplo de Contrato – POST /v1/query	14
8	Stack Tecnológica	15
9	Infraestrutura e HPC-Readiness	16
10	Estratégia de Testes	17
11	Considerações de Segurança	18
12	Roadmap e Trabalhos Futuros	19
A	Apêndice A – Registro de Decisão Arquitetural (ADR-01)	20

1 Introdução

1.1 Contexto

O projeto *Parliament Graph Architecture* (TCC) produziu uma pipeline reprodutível de análise de redes de coautoria parlamentar, validada estatisticamente contra modelos nulos (hipótese H1, $p < 0,005$ em quatro anos consecutivos). O TCC é intencionalmente fechado como um artefato científico: sua responsabilidade termina na geração de indicadores auditáveis (`analysis_{year}.json`, CSV de métricas por deputado, GEXF e SQLite).

O **Parliament Insights API** é um sistema *separado*, que **consome** esses artefatos como fonte de dados somente-leitura e adiciona uma camada de produto: uma API de backend tradicional (NestJS) combinada com uma camada de Inteligência Artificial Generativa (RAG, agentes e sumarização por LLM), sem jamais alterar o núcleo de validação estatística do TCC.

1.2 Objetivos

- Demonstrar competência de Engenharia de Software em um sistema que integra backend tradicional, orquestração de agentes de IA e infraestrutura de dados vetoriais.
- Servir como material de portfólio técnico alinhado a vagas de *Cientista de Dados / Engenheiro(a) de IA Generativa*, cobrindo arquitetura Transformer, RAG, agentes, e práticas de HPC-readiness (containerização, filas, separação de camadas de cômputo).
- Preservar a integridade metodológica do TCC, tratando-o como fonte de verdade imutável.

1.3 Escopo

Este documento cobre: requisitos funcionais e não-funcionais; casos de uso; arquitetura em camadas; modelo de dados; principais fluxos (diagramas de sequência); design de API; stack tecnológica; estratégia de testes; considerações de segurança e infraestrutura; e roadmap. Não cobre: a metodologia estatística do TCC (já documentada em seu próprio README), nem código-fonte de implementação (este é um documento de análise e projeto, anterior à codificação completa).

1.4 Público-Alvo

Recrutadores técnicos, avaliadores de portfólio, orientador acadêmico, e o próprio autor como referência de implementação.

2 Requisitos

2.1 Requisitos Funcionais

ID	Nome	Descrição
RF01	Consulta estruturada por métrica	O sistema deve permitir consultar, em linguagem natural, métricas estruturais já validadas (densidade, modularidade, centralidades) filtradas por ano, retornando o valor exato do dado de origem, sem geração numérica pelo LLM.
RF02	Busca semântica de proposições	O sistema deve permitir buscar proposições legislativas por similaridade de significado (não apenas palavra-chave), com filtro opcional por comunidade (Louvain) e ano.
RF03	Resumo temático de comunidade	O sistema deve gerar e disponibilizar um resumo textual do padrão temático de uma comunidade de deputados, com base nas ementas das proposições coautoradas por seus membros.
RF04	Comparação entre anos	O sistema deve permitir comparar um mesmo indicador (ex.: modularidade, densidade) entre múltiplos anos do período 2022–2025.
RF05	Roteamento de intenção	O sistema deve classificar automaticamente a intenção da pergunta do usuário (consulta estruturada, busca semântica, ou resumo) e direcioná-la à ferramenta apropriada.
RF06	Rastreabilidade de fontes	Toda resposta que envolva geração por LLM deve citar as proposições/documentos-fonte utilizados como contexto.
RF07	Cache de resumos	Resumos de comunidade já gerados devem ser cacheados e reaproveitados, evitando nova chamada ao LLM para a mesma comunidade/ano.
RF08	Autenticação e autorização	O acesso à API deve ser autenticado (API key ou JWT) com limitação de taxa (<i>rate limiting</i>) por cliente.
RF09	Ingestão de artefatos do TCC	O sistema deve importar periodicamente os artefatos gerados pelo pipeline do TCC (JSON/CSV/GEXF) para o banco de dados operacional, sem modificá-los na origem.
RF10	Processamento em lote de embeddings	A geração de embeddings das ementas das proposições deve poder ser executada de forma assíncrona/em lote, desacoplada do caminho de requisição síncrona da API.

2.2 Requisitos Não-Funcionais

ID	Categoria	Descrição
RNF01	Precisão / Confiabilidade	Respostas de consulta estruturada (RF01, RF04) devem ser deterministicamente corretas em relação ao dado de origem; o LLM atua apenas como camada de interpretação de linguagem, nunca como fonte do valor numérico.
RNF02	Desempenho	Consultas estruturadas devem responder em até 1s (p95); consultas RAG em até 5s (p95), incluindo chamada ao LLM.
RNF03	Escalabilidade	A camada de geração de embeddings e sumarização deve ser horizontalmente escalável via workers independentes, permitindo múltiplas instâncias sem estado compartilhado em memória.
RNF04	Portabilidade / HPC-readiness	Todos os serviços devem ser executáveis via container (Docker), stateless na camada de cômputo, e compatíveis com execução em lote (batch), refletindo os princípios de agendamento de cluster (ex.: SLURM).
RNF05	Observabilidade	Toda chamada ao LLM deve ser logada com: prompt, modelo utilizado, tokens consumidos, e latência, para fins de auditoria de custo.
RNF06	Segurança	Comunicação externa via HTTPS; segredos (chaves de API de LLM) gerenciados via variáveis de ambiente/ <i>secret manager</i> , nunca versionados.
RNF07	Manutenibilidade	Separação estrita de camadas (API / Orquestração / Ferramentas / Dados), seguindo o mesmo princípio de <i>Dependency Rule</i> já adotado no TCC.
RNF08	Testabilidade	Cobertura de testes automatizados $\geq 80\%$ nas camadas de API e de ferramentas (<i>tools</i>), com testes de integração cobrindo os três fluxos principais (RF01–RF03).
RNF09	Custo	A arquitetura deve permitir alternância entre LLM local (Ollama) e LLM via API paga, possibilitando comparação de custo por consulta.
RNF10	Integridade dos dados de origem	O sistema nunca deve escrever ou alterar os artefatos originais do TCC; toda ingestão é somente-leitura na origem.

3 Análise de Casos de Uso

3.1 Diagrama de Casos de Uso

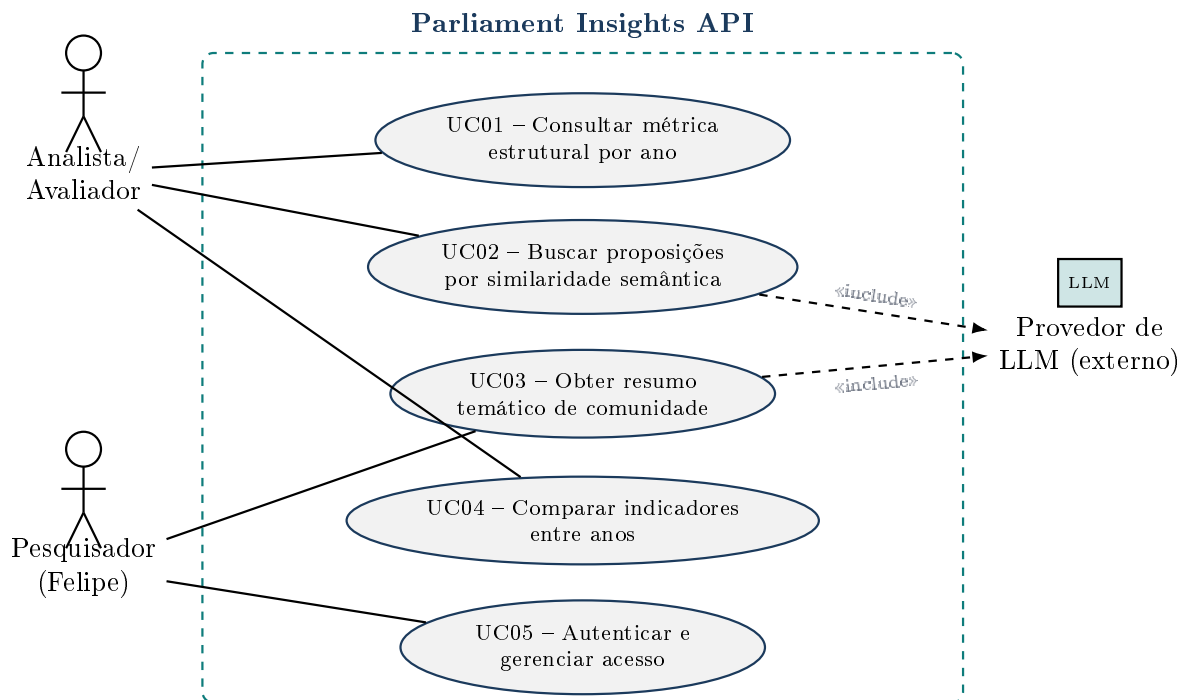


Figura 1: Casos de uso do Parliament Insights API

3.2 Especificação dos Casos de Uso Principais

UC01 – Consultar métrica estrutural por ano

Ator principal

Analista/Avaliador

Pré-condição

Artefatos do TCC já ingeridos na base operacional (RF09).

Fluxo principal 1. Usuário envia pergunta em linguagem natural (ex.: "qual foi a modularidade em 2024?").

2. Agente classifica a intenção como consulta estruturada (RF05).

3. *Structured Query Tool* extrai parâmetros (ano, métrica) e executa consulta parametrizada no banco.

4. Resultado é formatado em linguagem natural e devolvido com o valor exato.

Fluxo alternativo

Métrica ou ano inexistente → resposta explícita de "dados não disponíveis", sem inferência.

Pós-condição

Resposta correta e rastreável ao `analysis_{year}.json` de origem.

UC02 – Buscar proposições por similaridade semântica

Ator principal

Analista/Avaliador

Pré-condição

Embeddings das ementas já gerados (RF10).

Fluxo principal 1. Usuário faz pergunta temática (ex.: *"quais propostas sobre meio ambiente a comunidade 3 coautorou em 2025?"*).

2. Agente identifica intenção de busca semântica e extrai filtros (comunidade, ano).
3. *RAG Tool* gera embedding da pergunta e executa busca por similaridade (pgvector) com filtro de metadados.
4. Contexto recuperado é enviado ao LLM, que gera resposta citando as proposições-fonte (RF06).

Pós-condição

Resposta com citação explícita de `proposition_id` utilizados.

UC03 – Obter resumo temático de comunidade**Ator principal**

Pesquisador

Fluxo principal 1. Processo em lote (assíncrono) coleta as ementas de todas as proposições coautoradas por uma comunidade Louvain.

2. LLM gera resumo temático do conjunto.
3. Resumo é armazenado em cache (`CommunitySummary`).
4. API expõe o resumo já pronto sob demanda, sem nova chamada ao LLM (RF07).

Restrição

O resumo é rotulado explicitamente como ferramenta exploratória, não como parte da validação estatística do TCC.

4 Arquitetura de Software

4.1 Visão Geral em Camadas

A arquitetura segue o mesmo princípio de *Dependency Rule* adotado no TCC: cada camada depende apenas da camada imediatamente inferior, nunca o inverso, e o cruzamento entre tecnologias (Node.js ↔ Python) ocorre em uma única fronteira bem definida (API → Orquestração).

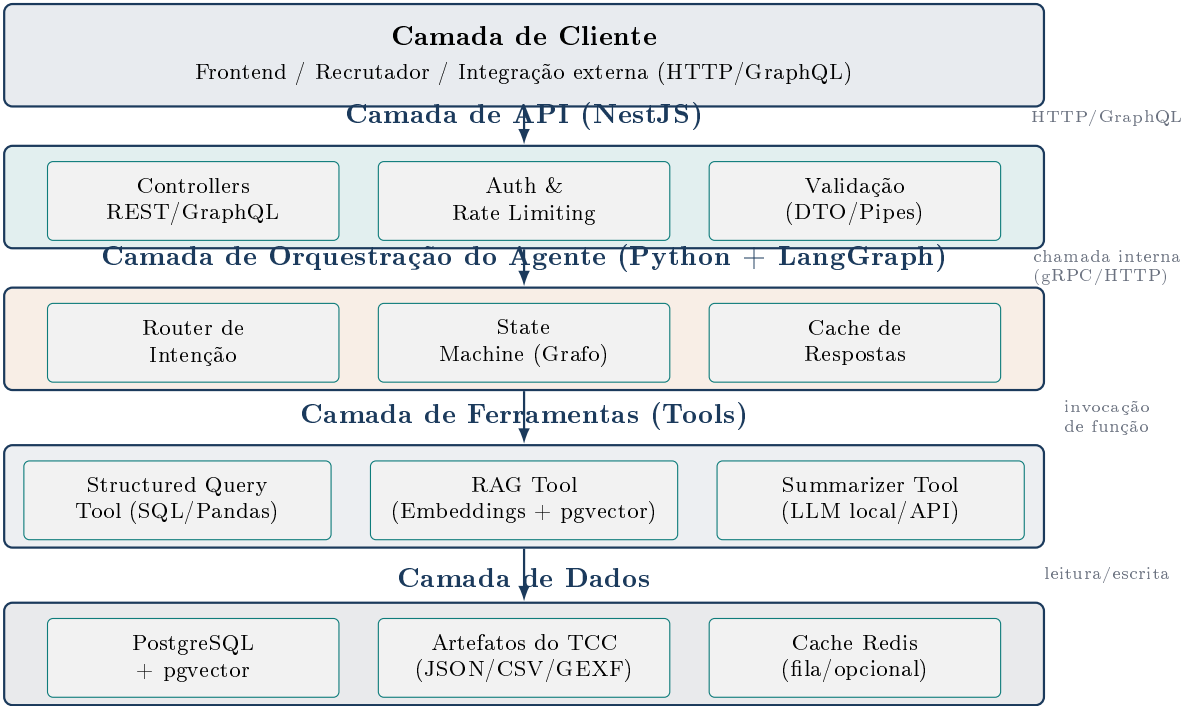


Figura 2: Arquitetura em camadas do Parliament Insights API

4.2 Descrição das Camadas

Camada	Responsabilidade
API (NestJS)	Exposição HTTP/GraphQL, autenticação, validação de entrada (DTOs), <i>rate limiting</i> , e tradução entre o contrato público da API e a chamada interna ao serviço de orquestração. Não contém lógica de IA.
Orquestração do Agente (Python + LangGraph)	Máquina de estados que representa o fluxo do agente: recebe a pergunta, classifica a intenção (RF05), decide qual(is) ferramenta(s) invocar, e agrega o resultado final. Não acessa o banco diretamente – delega às ferramentas.
Ferramentas (Tools)	Três implementações independentes e testáveis isoladamente: <i>Structured Query Tool</i> (determinístico), <i>RAG Tool</i> (busca semântica + geração), <i>Summarizer Tool</i> (sumarização em lote). Cada ferramenta encapsula uma responsabilidade única.
Dados	PostgreSQL com extensão <i>pgvector</i> para embeddings; artefatos do TCC como fonte de importação somente-leitura; Redis como fila opcional para processamento assíncrono.

4.3 Padrões e Decisões Arquiteturais

- **Monorepo com fronteiras de módulo claras** (`api/`, `agent/`, `data/`) em vez de múltiplos repositórios – ver ADR-01 (Apêndice A).
- **Separação entre geração de valor numérico e geração de linguagem**: a *Structured Query Tool* nunca delega ao LLM a computação de um número já validado pelo TCC – isso evita alucinação estatística (RNF01).
- **GraphRAG**: a busca semântica (RAG Tool) sempre combina similaridade vetorial *com* filtro estrutural pelos metadados do grafo (comunidade, ano), em vez de busca vetorial pura.
- **Processamento em lote desacoplado** para sumarização (RF10), preparando o sistema para o padrão de execução em cluster (fila de jobs análoga ao SLURM) exigido por vagas de IA/HPC.
- **Independência de provedor de LLM**: a camada de Ferramentas abstrai o provedor (Ollama local ou API paga) atrás de uma interface comum, permitindo comparação de custo/-qualidade (RNF09).

5 Modelo de Dados

5.1 Diagrama Entidade-Relacionamento

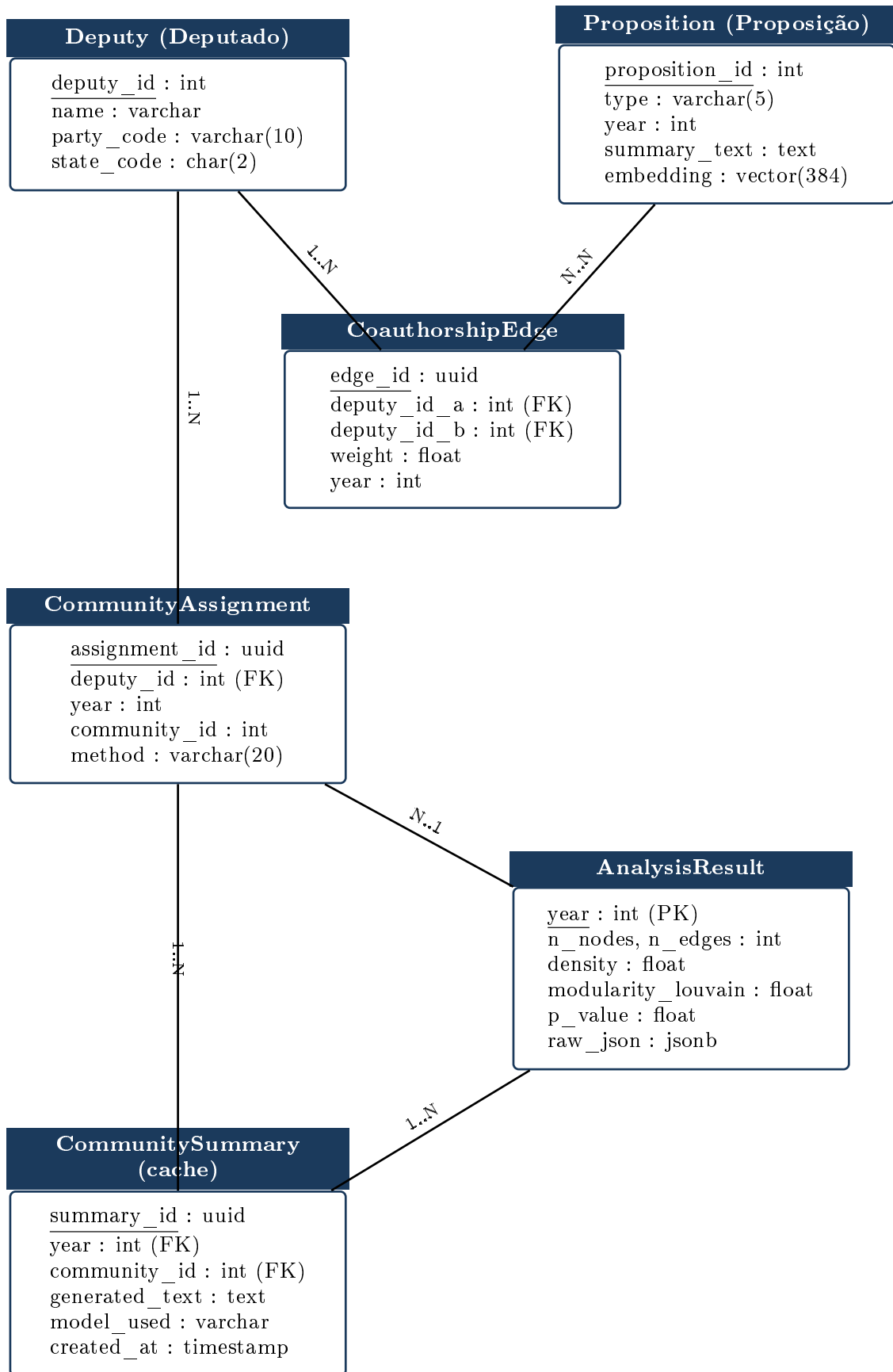


Figura 3: Modelo de dados operacional do Parliament Insights API

5.2 Dicionário de Dados (resumido)

Entidade	Origem	Observações
Deputy	Importado do TCC (CSV de métricas)	Chave natural <code>deputy_id</code> preservada do dado original da Câmara.
Proposition	Importado + enriquecido	Campo <code>embedding</code> (vetor de 384 dimensões, <code>sentence-transformers</code>) é gerado nesta camada, não existe no TCC.
CoauthorshipEdge	Importado do GEXF	Réplica normalizada das arestas do grafo de co-autoria para consulta relacional eficiente.
CommunityAssignment	Importado do CSV de métricas	Atribuição de comunidade Louvain/Label Propagation por deputado/ano.
AnalysisResult	Importado do <code>analysis_{year}.json</code>	Tabela append-only; nunca é reescrita, apenas versionada por ano – garante RNF10.
CommunitySummary	Gerado nesta camada	Cache de resultado de LLM; inclui <code>model_used</code> para auditoria de custo (RNF05).

6 Fluxos Principais (Diagramas de Sequência)

6.1 Fluxo 1 – Consulta Estruturada (UC01)

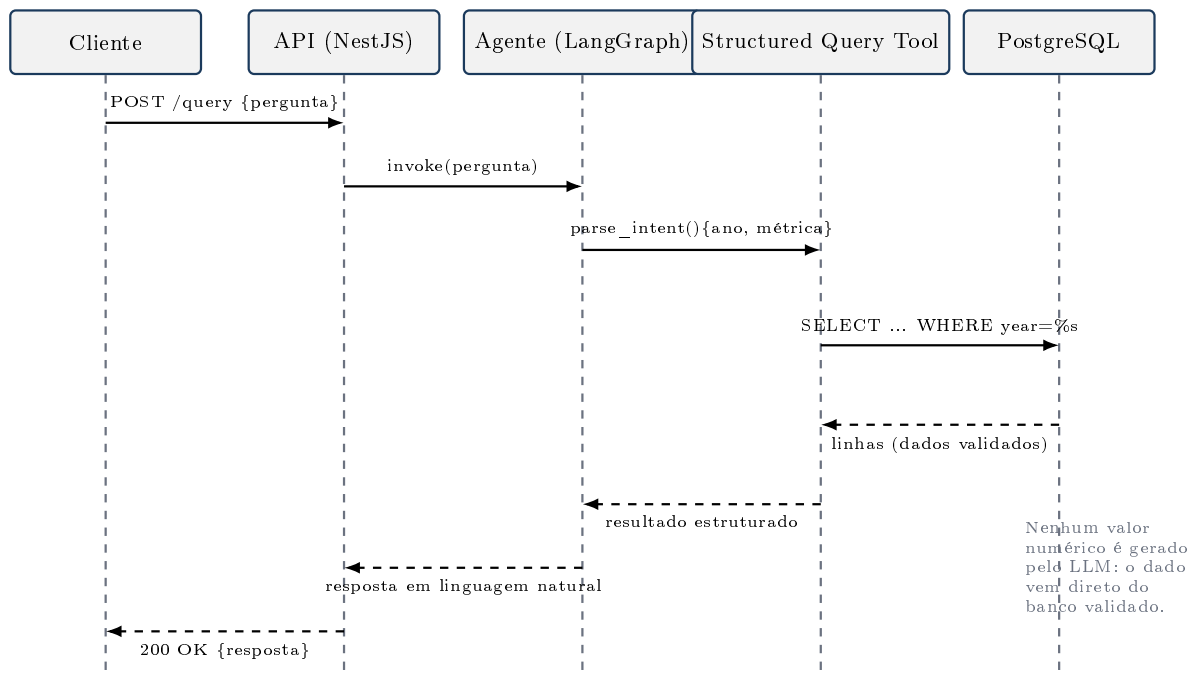


Figura 4: Sequência de execução de uma consulta estruturada

6.2 Fluxo 2 – Busca RAG (UC02)

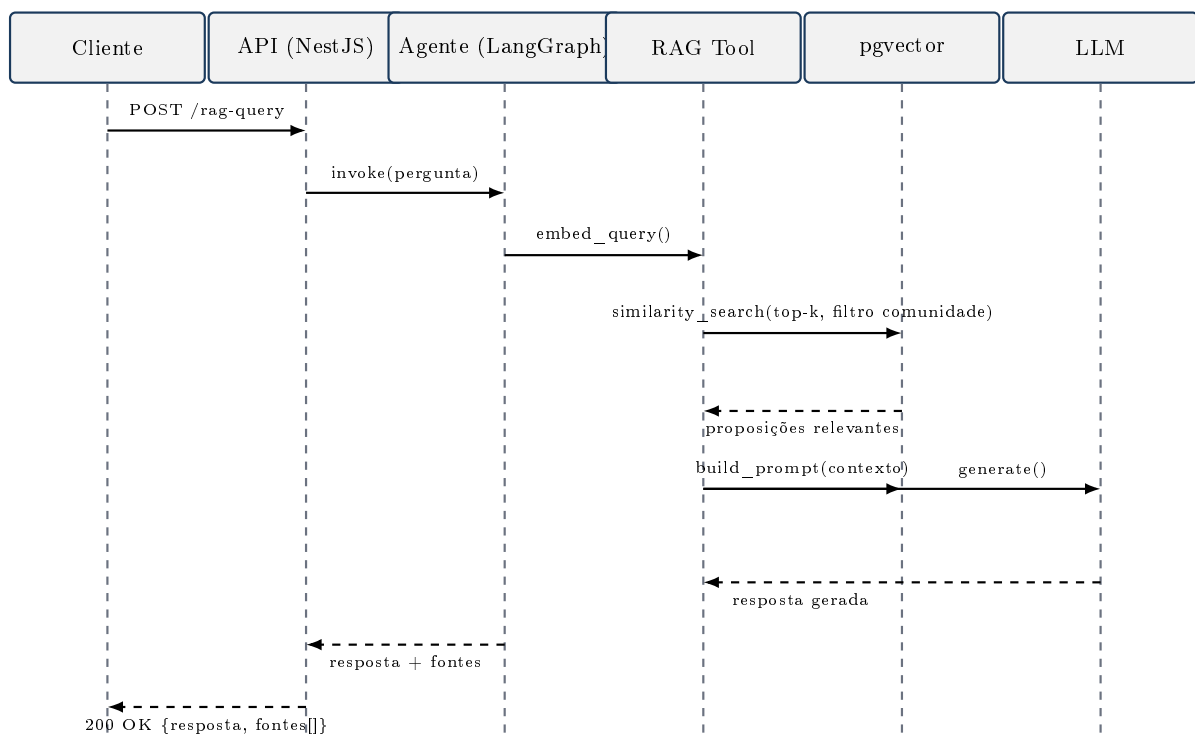


Figura 5: Sequência de execução de uma busca semântica com geração aumentada

6.3 Fluxo 3 – Sumarização em Lote (UC03)

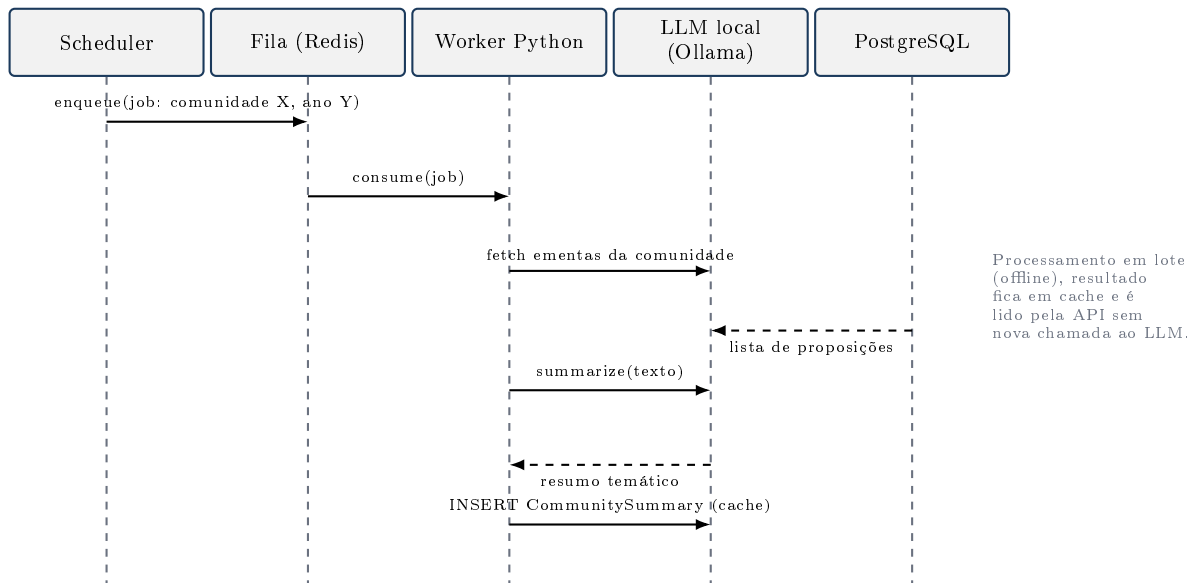


Figura 6: Sequência de execução assíncrona do resumo de comunidade

7 Design de API

7.1 Endpoints REST

Método	Rota	Descrição
POST	/v1/query	Ponto de entrada único do agente; roteia para RF01, RF02 ou RF03 conforme a intenção classificada.
GET	/v1/metrics/{year}	Consulta estruturada direta (bypass do agente), retorna o <code>AnalysisResult</code> bruto do ano.
GET	/v1/metrics/compare?years=2022,2025&metric=modularity	Comparação entre anos (RF04).
POST	/v1/search	Busca semântica direta (RF02) sem envolver geração textual – retorna apenas os documentos.
GET	/v1/communities/{year}/{id}/summary	Recupera resumo de comunidade já cacheado (RF03/RF07); 202 Accepted se ainda em processamento.
POST	/v1/admin/ingest	Dispara a ingestão dos artefatos do TCC (RF09); restrito a papel administrativo.

7.2 Exemplo de Contrato – POST /v1/query

Requisição:

```
{
  "question": "Qual foi a modularidade da rede em 2024?",
  "context": { "preferred_language": "pt-BR" }
}
```

Resposta (200):

```
{
  "intent": "structured_query",
  "answer": "Em 2024, a modularidade (Louvain) observada foi 0.634, com p-valor 0.005 (significativo).",
  "source": {
    "type": "AnalysisResult",
    "year": 2024,
    "field": "louvain.modularity"
  },
  "generated_by_llm": false,
  "latency_ms": 340
}
```

Resposta (200) – intenção RAG:

```
{
  "intent": "semantic_search",
  "answer": "A comunidade 3 coautorou 4 proposicoes sobre meio ambiente em 2025...",
  "sources": [
    { "proposition_id": 88213, "type": "PL", "year": 2025, "score": 0.87 },
    { "proposition_id": 88540, "type": "PEC", "year": 2025, "score": 0.81 }
  ],
  "generated_by_llm": true,
  "model_used": "llama3.2:3b",
  "latency_ms": 2130
}
```

8 Stack Tecnológica

Camada		Tecnologia	Justificativa
API	HTTP/-	NestJS (TypeScript)	Stack já dominada pelo autor; tipagem forte e módulos alinhados ao princípio de camadas.
Orquestração de agente		Python + LangGraph	Ecosistema padrão de mercado para orquestração de agentes com estado explícito (grafo de decisão).
Embeddings		sentence-transformers	Execução local, sem custo de API, controle total sobre a dimensionalidade do vetor.
Vector store		PostgreSQL + pgvector	Reaproveita o mesmo SGBD relacional já usado, evitando operar um banco vetorial dedicado.
LLM local		Ollama (Llama 3.2 3B)	Permite comparação de custo/qualidade com API paga (RNF09) e demonstra domínio de deploy local de modelo.
Fila / processamento assíncrono		Redis + BullMQ	Desacopla geração de embeddings/sumarização do caminho síncrono da API (RF10).
Containerização		Docker + Docker Compose	Mesma abordagem do TCC; caminho direto para HPC-readiness (RNF04).

9 Infraestrutura e HPC-Readiness

Embora o escopo deste projeto não exija um cluster HPC real, a arquitetura é desenhada para refletir os princípios que tornam um sistema *pronto para HPC*, atendendo ao requisito "desejável" de conhecimento em SLURM/containers da vaga de referência:

- **Stateless na camada de cômputo:** o *worker* de embeddings/sumarização não mantém estado em memória entre execuções – cada job é independente, análogo a um *job array* do SLURM.
- **Batch-first:** a geração de embeddings roda em lote sobre todas as proposições pendentes, não uma a uma sob demanda.
- **Containerização:** todo serviço roda em imagem Docker isolada; a migração para Apptainer/Singularity (comum em ambientes acadêmicos de HPC) exigiria apenas troca do executor de container, não do código.
- **Fila como abstração de escalonador:** a fila Redis/BullMQ cumpre, em escala reduzida, o mesmo papel que o escalonador SLURM cumpre em um cluster – distribuir jobs entre workers disponíveis.

10 Estratégia de Testes

Nível	Abordagem
Unitário	<i>Structured Query Tool</i> : 100% das combinações de parâmetros de consulta testadas com banco em memória/ <i>fixture</i> . <i>RAG Tool</i> : mock do provedor de embeddings e do vector store.
Integração	Fluxo completo API → Agente → Ferramenta → Banco, para os três casos de uso principais (UC01–UC03), com banco de teste populado a partir de uma amostra fixa dos artefatos do TCC.
Contrato	Validação de schema de requisição/resposta de cada endpoint (RF08) via testes de contrato automatizados.
Qualidade de geração	Para RF02/RF03, avaliação amostral manual (não automatizável em CI) da relevância dos documentos recuperados e da fidelidade do resumo gerado às ementas-fonte.
Regressão de custo	Testes que registram tokens/latência por chamada ao LLM, alertando se a média por consulta ultrapassar um limiar definido (RNF05/RNF09).

11 Considerações de Segurança

- Chaves de API de provedores de LLM externos nunca versionadas – gerenciadas via variáveis de ambiente/`.env` (seguindo o padrão já usado no TCC com `pydantic-settings`).
- *Rate limiting* por chave de API para mitigar uso abusivo e conter custo de chamadas ao LLM externo.
- Sanitização de entrada antes de compor prompts, mitigando injeção de prompt via texto de proposições ou perguntas do usuário.
- Endpoint de ingestão (`/v1/admin/ingest`) restrito por papel administrativo, nunca exposto publicamente.
- Dados de origem (TCC) tratados como *read-only* em todo o pipeline de ingestão – nenhuma escrita reversa é permitida (RNF10).

12 Roadmap e Trabalhos Futuros

1. **V1 (MVP):** *Structured Query Tool* + ingestão dos artefatos do TCC + endpoint `/v1/metrics`.
2. **V2:** RAG sobre ementas de proposições + endpoint `/v1/search` e `/v1/query`.
3. **V3:** Sumarização de comunidades em lote + cache + endpoint `/v1/communities`.
4. **V4 (extensão):** Fine-tuning leve (LoRA/PEFT) de um modelo pequeno especializado no domínio legislativo, comparando qualidade/custo contra o Ollama genérico – ponto de conexão direta com a ideia de consultoria de LLMs locais.
5. **V5 (extensão):** Extração do módulo **agent**/ como pacote Python reutilizável, caso surjam outros projetos de grafo que se beneficiem da mesma camada de RAG/agente (ver Seção 6.3).

A Apêndice A – Registro de Decisão Arquitetural (ADR-01)

Campo	Conteúdo
Título	Monorepo vs. múltiplos repositórios para as camadas API/Agente/Dados
Status	Aceito
Contexto	O projeto tem duas linguagens (TypeScript/Python) e quatro camadas lógicas. É um projeto de portfólio individual, não um sistema com múltiplos times.
Decisão	Adotar monorepo com pastas api/ , agent/ , data/ , comunicando-se via HTTP interno, em vez de repositórios separados por camada.
Alternativas consideradas	(1) Poli-repo com um repositório por camada; (2) Monorepo com <i>workspaces</i> (npm/uv) integrados.
Consequências positivas	Leitura unificada da arquitetura por um avaliador externo; menor sobrecarga de versionamento cruzado; deploy único via docker-compose .
Consequências negativas	Menor isolamento de <i>ownership</i> por camada (irrelevante no contexto de autor único); caso o módulo agent/ precise virar biblioteca reutilizável (Seção 10, item V5), será extraído para repositório próprio nesse momento.